



LAB INTRO



Lab Organization

3 Weeks

Part 1

- Environment setup
- Using a pre trained model
- Familiarizing with Libraries
- Using/Calculating metrics

4 Weeks

Part 2

- Mining vulnerabilities
- Extracting features
- Labeling the data
- Create your own dataset

3 Weeks

Part 3

- Build your own ML pipeline
- Use dataset from part 2
- Evaluate performance of your model

Project Presentation

General information:

- Groups of 3
- Discussion in each group during/after each part
- Bonus points for project presentation

Software recommendations:

- Python 3
- Jupyter Notebook
- VS Code
 - jupyter notebook plugin



INTRODUCTION: PYTORCH

Quick Facts

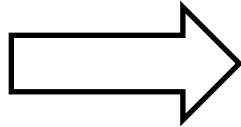
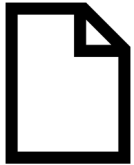
- Developed in 2016 by Facebook (now Meta AI)
- Powerful machine learning framework (especially deep learning)
- Free and Open Source
- Bindings in Python, Backend in C++
- Widely used in research and industry
 - E.g. Tesla's Autopilot, Uber's Pyro, Hugging Face's etc.



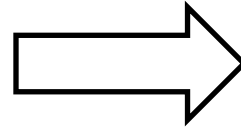
Steps of a typical ML Pipeline

Training:

Dataset loading



Training



Evaluation

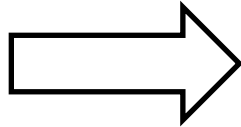
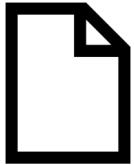


- This course focuses on training and evaluating ML pipelines
- Resources on using ML pipelines in production an environment can be found online: e.g.: <https://pytorch.org/blog/model-serving-in-pyorch/>

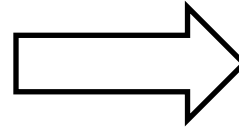
Steps of a typical ML Pipeline

Training:

Dataset loading



Training



Evaluation



Loading Datasets

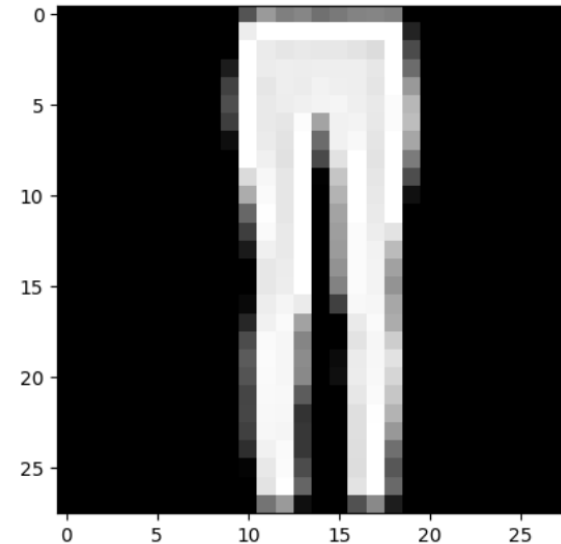
- PyTorch contains a variety of pre-defined dataloaders for existing datasets
- Custom dataloaders can easily be implemented(Lab)

```
# Download training data from open datasets.  
training_data = datasets.FashionMNIST(  
    root="data",  
    train=True,  
    download=True,  
    transform=ToTensor(),  
)
```

```
# Download test data from open datasets.  
test_data = datasets.FashionMNIST(  
    root="data",  
    train=False,  
    download=True,  
    transform=ToTensor(),  
)
```

Exploring the datasets

```
# Display image and label.
train_features, train_labels = next(iter(train_dataloader))
print(f"Feature batch shape: {train_features.size()}")
print(f"Labels batch shape: {train_labels.size()}")
img = train_features[0].squeeze()
label = train_labels[0]
plt.imshow(img, cmap="gray")
plt.show()
print(f"Label: {label}")
```



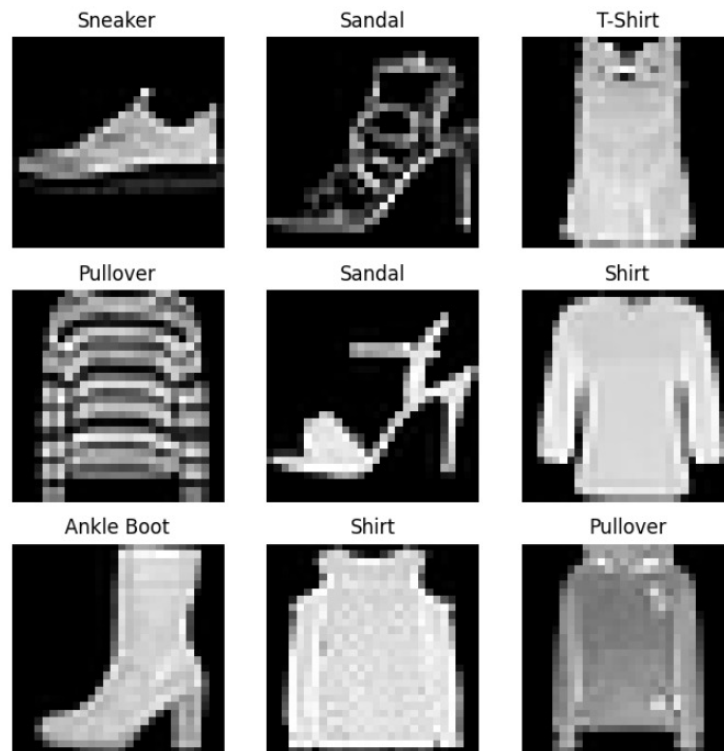
Out:

```
Feature batch shape: torch.Size([64, 1, 28, 28])
Labels batch shape: torch.Size([64])
Label: 1
```

Iterating and visualizing a dataset

```
labels_map = {
    0: "T-Shirt",
    1: "Trouser",
    2: "Pullover",
    3: "Dress",
    4: "Coat",
    5: "Sandal",
    6: "Shirt",
    7: "Sneaker",
    8: "Bag",
    9: "Ankle Boot",
}

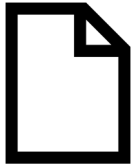
figure = plt.figure(figsize=(8, 8))
cols, rows = 3, 3
for i in range(1, cols * rows + 1):
    sample_idx = torch.randint(len(training_data), size=(1,)).item()
    img, label = training_data[sample_idx]
    figure.add_subplot(rows, cols, i)
    plt.title(labels_map[label])
    plt.axis("off")
    plt.imshow(img.squeeze(), cmap="gray")
plt.show()
```



Steps of a typical ML Pipeline

Training:

Dataset loading



Training



Evaluation



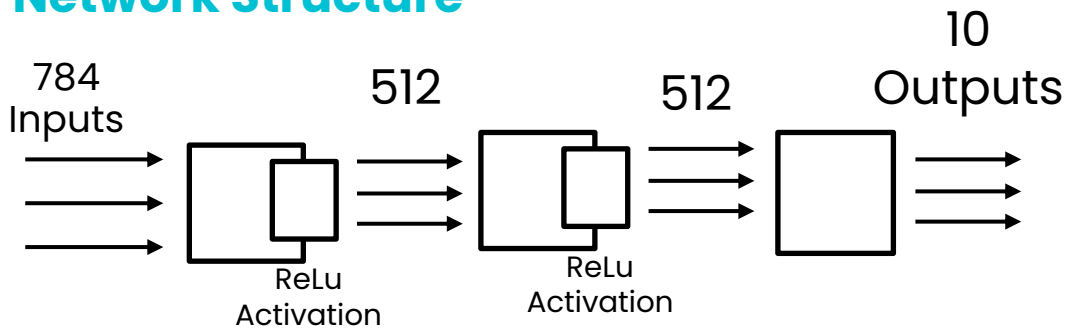
Define a model

```
# Define model
class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28*28, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 10)
        )

    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)
        return logits

model = NeuralNetwork().to(device)
print(model)
```

Network Structure



flatten()

```
>>> t = torch.tensor([[[[1, 2],
...                      [3, 4]],
...                      [[5, 6],
...                      [7, 8]]]])
>>> torch.flatten(t)
tensor([1, 2, 3, 4, 5, 6, 7, 8])
```

Training a model

- Steps involved in training:
 1. Set model into training mode (weights can be changed)
 2. Load a batch from the dataset
 3. Predict a value
 4. Calculate the loss compared to the label
 5. Update weights with backpropagation

```
loss_fn = nn.CrossEntropyLoss()  
optimizer = torch.optim.SGD(model.parameters(), lr=1e-3)
```

```
def train(dataloader, model, loss_fn, optimizer):  
    size = len(dataloader.dataset)  
    model.train()  
    for batch, (X, y) in enumerate(dataloader):  
        X, y = X.to(device), y.to(device)  
  
        # Compute prediction error  
        pred = model(X)  
        loss = loss_fn(pred, y)  
  
        # Backpropagation  
        optimizer.zero_grad()  
        loss.backward()  
        optimizer.step()  
  
    if batch % 100 == 0:  
        loss, current = loss.item(), (batch + 1) * len(X)  
        print(f"loss: {loss:>7f} [{current:>5d}/{size:>5d}]")
```



Sample Training Output

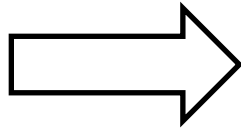
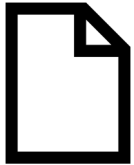
```
Epoch 5
-----
loss: 1.327389 [ 64/60000]
loss: 1.311585 [ 6464/60000]
loss: 1.147247 [12864/60000]
loss: 1.249725 [19264/60000]
loss: 1.136566 [25664/60000]
loss: 1.145563 [32064/60000]
loss: 1.179174 [38464/60000]
loss: 1.113521 [44864/60000]
loss: 1.149795 [51264/60000]
loss: 1.063133 [57664/60000]
Test Error:
  Accuracy: 65.1%, Avg loss: 1.085360

Done!
```

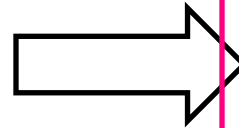
Steps of a typical ML Pipeline

Training:

Dataset loading



Training



Evaluation



Evaluating a Model

```
def test(dataloader, model, loss_fn):
    size = len(dataloader.dataset)
    num_batches = len(dataloader)
    model.eval()
    test_loss, correct = 0, 0
    with torch.no_grad():
        for X, y in dataloader:
            X, y = X.to(device), y.to(device)
            pred = model(X)
            test_loss += loss_fn(pred, y).item()
            correct += (pred.argmax(1) == y).type(torch.float).sum().item()
    test_loss /= num_batches
    correct /= size
    print(f"Test Error: \n Accuracy: {(100*correct):>0.1f}%, Avg loss: {test_loss:>8f} \n")
```

Saving a model

- Saving a model is straight forward
- A saved model is referred to as
“serialized”
- Note: It is common to save PyTorch models as “.pt” or “.pth” files

```
torch.save(model.state_dict(), "model.pth")  
print("Saved PyTorch Model State to model.pth")
```

Loading a Model

```
model = NeuralNetwork()  
model.load_state_dict(torch.load("model.pth"))
```

Out:

```
<All keys matched successfully>
```

Using the model for prediction

1. Set the model to **evaluation mode**
2. Disable gradient calculation (best practice)
3. Feed input values to the model
4. Prediction is returned

```
classes = [  
    "T-shirt/top",  
    "Trouser",  
    "Pullover",  
    "Dress",  
    "Coat",  
    "Sandal",  
    "Shirt",  
    "Sneaker",  
    "Bag",  
    "Ankle boot",  
]  
  
model.eval()  
x, y = test_data[0][0], test_data[0][1]  
with torch.no_grad():  
    pred = model(x)  
    predicted, actual = classes[pred[0].argmax(0)], classes[y]  
    print(f'Predicted: "{predicted}", Actual: "{actual}"')
```

Out:

Predicted: "Ankle boot", Actual: "Ankle boot"

Tips

- Check for GPU support
 - GPUs/MPSSs are beneficial for ML applications since they are optimized for repetitive parallel execution of instructions

```
# Get cpu or gpu device for training.  
device = "cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available()  
else "cpu"  
print(f"Using {device} device")
```

Useful Libraries

- Familiarize with the following libraries:
 - Pandas
 - https://pandas.pydata.org/docs/user_guide/10min.html
 - Numpy
 - <https://numpy.org/doc/stable/user/quickstart.html>
 - Matplotlib
 - https://matplotlib.org/stable/tutorials/introductory/quick_start.html#sphx-glr-tutorials-introductory-quick-start-py